ALI HAIDER | Enrollment: 09-131242-016

MUSTAFA HUSSAIN SOMORO | Enrollment: 09-131242

Software Construction Project Proposal

SportsWeek Manager: A Paperless Sports Week Management and Fixture Automation System

Project Proposal, prepared in the style of IEEE Std 1058 (SPMP), IEEE/ISO 29148 (Requirements) and IEEE 1016 (Design)

Document ID	SW-PP-001
Version	1.0 (Draft for approval)
Date	24 September 2026
Project type	Web application (responsive) with REST API, real-time updates

Abstract

During sports week, organizers handle team registration, attendance checks, match schedules, score sheets and result tables on paper. This is slow, error-prone and prone to schedule clashes. This proposal describes **SportsWeek Manager**, a system in which students register teams online, and organizers create tournaments for **Tug of War, Cricket, Futsal, Chess, Badminton and Table Tennis**, generate knockout fixtures, auto-schedule matches without venue, team or player clashes, record scores, handle withdrawals and walkovers, and publish live leaderboards by sport and by department. The key technical challenge is a **dynamic fixture engine** that keeps the bracket correct when teams withdraw, matches end in walkovers, or byes occur. It is built test-first and validated with simulated tournaments of every size.

Index Terms: sports management, tournament fixtures, single elimination, auto-scheduling, conflict detection, leaderboard, ASP.NET Core, SQL Server, SignalR.

I. Introduction

A. Background

College sports weeks involve many sports, venues and hundreds of participants over a few days. Manual coordination causes lost registrations, double-booked grounds, players scheduled in two matches at once, and disputes over results.

B. Purpose

To seek approval and resources to design, build, test and deploy SportsWeek Manager before the next sports week.

C. Definitions

Term	Meaning
Fixture	A scheduled match between two participants
Bracket	The full knockout tree of a tournament
Bye	An automatic advance to the next round without playing
Walkover (WO)	A win awarded because the opponent withdrew or did not arrive
Double forfeit	Both participants withdraw; neither advances
Clash	A scheduling conflict (venue, team, player or time rule)

II. Problem Statement

1. Registration is on paper; teams cannot be verified quickly.
2. Fixtures are drawn by hand and break whenever a team withdraws.
3. Manual schedules cause venue, team and player clashes.
4. Results and points are compiled manually; standings are late and disputed.
5. Students do not know match times; last-minute changes are not communicated.

III. Objectives

ID	Objective	Measure of success
O1	Online team registration with validation	100% of teams registered online; no duplicate players in the same sport
O2	Automatic bracket generation for any team count (2 to 64)	Correct bracket for all counts in the test suite
O3	Clash-free auto-scheduling and safe manual rescheduling	0 undetected clashes across simulated runs
O4	Correct handling of withdrawals, walkovers, byes and double forfeits	All scenario tests pass
O5	Live results and leaderboards (per sport and per department)	Update within 5 seconds of score entry
O6	Upcoming-match notifications	Delivered to registered teams at least 30 minutes before the match
O7	Remove paper score sheets and registers	Fully digital operation during the pilot

IV. Scope

A. In scope

- Sports: Tug of War, Cricket, Futsal, Chess, Badminton, Table Tennis (admin can add more).
- Admin portal: sports and tournaments, registration approval, fixture generation, auto/manual scheduling, check-in, score entry, withdrawals, leaderboards, reports.
- Student portal: registration with department, semester and enrolment number; team registration; schedules; brackets; leaderboards; notifications.
- Single-elimination knockout format with byes and optional third-place match.
- Audit trail, role-based access, deployment and training.

B. Out of scope (v1)

Native mobile apps, payments, live streaming, ball-by-ball cricket scoring, double-elimination and league tables (planned as v1.1 and v2).

C. Assumptions

Students hold a valid enrolment number; the college provides venue lists and the sports-week date window; organizers enter scores from a tablet or laptop.

V. Existing versus Proposed System

Aspect	Existing (paper)	Proposed
Registration	Forms and lists	Online with validation and approval
Fixtures	Hand-drawn	Auto-generated and self-updating
Scheduling	Manual, clashes common	Auto scheduler plus real-time clash check
Withdrawals	Redraw by hand	Automatic walkover and advancement
Results	Score sheets	Live scores and leaderboards

Aspect	Existing (paper)	Proposed
Communication	Notice board	Dashboard notifications

VI. Functional Requirements (summary; details in 01_PRD)

ID	Requirement	Priority
FR-1	Users register with department, semester, enrolment number and log in.	High
FR-2	Captains register teams for open tournaments; the system validates player eligibility.	High
FR-3	Admin approves or rejects registrations and closes registration.	High
FR-4	Admin creates sports and tournaments and configures rules per sport.	High
FR-5	System generates a knockout bracket with byes for any team count.	High
FR-6	System auto-schedules matches and prevents clashes; admin can override with a check.	High
FR-7	Admin can reschedule; the system validates against venue, team, player and round-order rules.	High
FR-8	Admin checks teams in, marks withdrawals, and the system awards walkovers and advances winners.	High
FR-9	Scorers enter results per sport; winners advance automatically.	High
FR-10	Leaderboards per sport and overall department ranking.	High
FR-11	Dashboard shows upcoming matches and notifications.	Medium
FR-12	Re-draw of not-yet-played pairings when the field changes.	Medium

VII. Non-Functional Requirements

Category	Requirement
Performance	Bracket generation for 64 teams < 1 s; auto-schedule of a full tournament < 10 s; p95 API < 500 ms
Concurrency	Two admins cannot overwrite each other's schedule change (optimistic concurrency)
Security	HTTPS, hashed passwords, RBAC (Admin, Scorer, Captain), audit of all admin actions
Reliability	99% uptime during sports week; automated backups; offline-tolerant score entry retries
Usability	Works on phones, tablets and laptops; core actions in three clicks
Data integrity	A match result can be corrected only through a controlled "re-open" that rolls back downstream advancement safely

VIII. Proposed Architecture

```
Student Portal (Web/PWA)  --\
Admin Portal (Web)         ---> ASP.NET Core Web API

Web API --> Fixture Engine --> SQL Server
Web API --> Scheduler and Clash Checker --> SQL Server
Web API --> Leaderboard Service --> SQL Server
Web API --> SignalR Hub (live updates) --> Portals
Web API --> Background Jobs (reminders, check-in timeouts)
```

The **Fixture Engine**, **Scheduler** and **Clash Checker** are pure, separately tested domain modules so they can be verified with thousands of simulated tournaments.

IX. Methodology

Iterative Agile (Scrum), 2-week sprints. The fixture engine and scheduler are built **first and test-first** (unit, property-based and simulation tests) because every other feature depends on them.

X. Work Plan

Phase	Weeks	Deliverable
Initiation, requirements, design	1 to 2	Approved PRD, TRD, wireframes
Foundation (auth, sports, registration)	3 to 5	Registration flow live on staging
Fixture engine and tests	4 to 8	Verified bracket and progression engine
Scheduler and clash checker	7 to 10	Auto/manual scheduling with clash detection
Scoring, leaderboards, notifications	9 to 12	Live results and rankings
Hardening, UAT, dry-run event	13 to 14	Test report, pilot sign-off
Deployment and training	14	Go-live before sports week

Roles: Project Manager/BA, backend developers (C#), frontend developer, QA engineer, UX designer (part-time), sports committee representative (client).

XI. Verification and Validation

Level	Approach
Unit	xUnit for engine, scheduler, clash rules, scoring
Property-based	FsCheck: invariants across random team counts, results and withdrawals
Simulation	10,000 randomized tournaments per build (sizes 2 to 64) with random withdrawals
Integration	API and database tests with concurrency scenarios
UAT	Mock sports week with real organizers
Security and load	OWASP checks, 300 concurrent users

Acceptance: all High requirements pass; simulation suite green; no open critical defects; organizer sign-off.

XII. Risk Management

Risk	Probability	Impact	Mitigation
Fixture logic errors after withdrawals	Medium	High	Test-first engine, simulations, state invariants
Scheduling clashes from manual edits	Medium	High	Central clash checker, no bypass except audited override
Last-minute rule changes by committee	High	Medium	Rules stored as configuration per sport
Organizers enter wrong scores	Medium	Medium	Confirm dialogs, re-open with audit
Poor connectivity at venues	Medium	Medium	Retry queue for score entry, lightweight pages
Low student adoption	Medium	Medium	QR codes and posters, department representatives

XIII. Resources and Budget

Effort is about 6 person-months over 14 weeks. Costs are hosting, domain, email/SMS or push service, and test devices. Free tiers are sufficient for the pilot; figures should be adapted to institutional rates. A contingency of 10 to 15% is recommended.

XIV. Deliverables

Approved PRD, TRD, flow, schema and UI/UX documents; source code; fixture-engine test suite and simulation report; deployed system; admin and student manuals; training session.

XV. Limitations and Future Work

Single elimination only in v1. Future: round-robin and group-plus-knockout formats, double elimination, ball-by-ball cricket scoring, SMS/WhatsApp alerts, live streaming links, Android app, QR-code attendance and certificate generation.

XVI. Conclusion

SportsWeek Manager replaces paperwork with a single reliable source of truth for registrations, fixtures, schedules and results. Its distinguishing strength is a rigorously tested fixture and scheduling engine that stays correct when the real world changes. Approval is requested to begin the initiation phase.

References

[1] *IEEE Standard for Software Project Management Plans*, IEEE Std 1058-1998.

[2] *ISO/IEC/IEEE 29148:2018, Requirements Engineering*.

[3] *IEEE Recommended Practice for Software Design Descriptions*, IEEE Std 1016-2009.

[4] Microsoft, "ASP.NET Core Documentation." https://learn.microsoft.com/aspnet/core

[5] Microsoft, "Entity Framework Core Documentation." https://learn.microsoft.com/ef/core

[6] K. Schwaber and J. Sutherland, "The Scrum Guide," 2020.

Approval

Role	Name	Signature	Date
Sponsor			
Project Manager			
Sports Committee Representative			